

Reference Checker App Group: Building an AI-Powered Citation Fact-Checker

Possible framing: Trust but Verify: Building an App to Catch Fabricated and Misused References in the Age of AI

This group will build a web application that checks whether references in a manuscript are real and whether citations are properly supported. The app accepts a Word document, verifies references against publication databases, and uses AI to fact-check citations against uploaded full-text PDFs. Built using Claude Code — no coding experience needed, you build by typing plain English prompts.

Ground rules: Document every prompt in the shared Google Doc. Test with intention — try to break the app. When something breaks, describe it to Claude Code in plain English. The AI won't be perfect and that's fine — this is a screening tool, not a verdict.

Date	Format	Main goal	Minimum goal	Stretch goal
Wed Jun 10	In person	Introductions	Explain the project, the problem of fabricated references, and final goals	
Mon Jun 15	Email check-in	Get set up	Log into Claude, try a practice prompt, confirm access works	Read through Phase 1 prompts and note questions
Wed Jun 17	In person	Build Phase 1: reference extraction	Upload a Word document, extract and display all references	Parse author, title, journal, year, DOI from each reference
Mon Jun 22	Email check-in	Test Phase 1	Test with different manuscripts, document what breaks	Fix edge cases and add CSV download
Wed Jun 24	In person	Build Phase 2: reference verification	Connect CrossRef and PubMed APIs, verify each reference is real	Add color-coded results: green/yellow/red with match details
Mon Jun 29	Email check-in	Test verification	Test with real references and 2–3 fake ones mixed in	Improve matching logic for missed references
Wed Jul 1	In person	Solidify Phases 1–2	Fix remaining bugs, app reliably extracts and verifies references	Add downloadable verification report and progress bar
Mon Jul 6	Email check-in	Prepare for Phase 3	Set up free Gemini API key, confirm Phases 1–2 are solid	Review Phase 3 prompts as a team

Wed Jul 8	In person	Build Phase 3: PDF upload	Upload full-text PDFs, app matches each PDF to its reference	Add manual matching dropdown for mismatches
Mon Jul 13	Email check-in	Test PDF matching	Confirm PDFs match to correct references, fix mismatches	Test with different PDF formats and lengths
Wed Jul 15	In person	Build Phase 3: AI citation checking	AI reads manuscript claims and matching PDFs, flags unsupported citations	Add summary stats and per- citation spot-check buttons
Mon Jul 20	Email check-in	Test AI accuracy	Run on a known manuscript, document false positives and negatives	Refine AI prompt for better accuracy
Wed Jul 22	In person	Build Phase 4: multi-user and feedback	Add login system, saved results, thumbs up/down feedback	Add admin dashboard with usage and accuracy stats
Mon Jul 27	Email check-in	Final testing	Full end-to-end test by every team member	Write a brief user guide for new users
Wed Jul 29	In person	Final polish	Fix last bugs, clean up interface	Record a screen-capture demo of the app
Fri Jul 31	Deadline	Final products	Working app: upload Word doc → verify references → color- coded report	Full app with AI citation checking, PDF upload, multi-user access, and feedback system

Full prompt guide from Claude – [Claude chat link](#)

Claude username

Overview

This guide contains the prompts you'll paste into Claude Code to build your Reference Checker app, phase by phase. After each phase, test the app thoroughly before moving on. If something doesn't work right, describe the problem to Claude Code in plain English and it will fix it.

Tip: Keep a Google Doc where you paste every prompt and note what happened. This is your project log.

PHASE 1: Upload a Word Document and Extract References

Goal: A working app where you upload a .docx file and it pulls out the reference list.

Prompt 1A — Initial App Setup

Paste this into Claude Code:

Build me a web app called "Reference Checker" with the following features:

1. A clean, professional landing page with the title "Reference Checker: Verify Your Manuscript References" and a brief description: "Upload your manuscript to verify that all references are real publications and properly cited."
2. A file upload area where users can drag and drop or browse to upload a Microsoft Word (.docx) file.
3. When a Word document is uploaded, the app should:
 - Parse the document and find the References or Bibliography section (it's usually at the end of the document, under a heading like "References", "Bibliography", or "Works Cited")
 - Extract each individual reference as a separate item
 - Display all extracted references in a numbered list on the screen
 - For each reference, try to identify: author(s), title, journal name, year, volume, pages, and DOI if present
4. Use a simple, modern UI with a blue and white color scheme. Make it mobile-friendly.
5. Use Python (Flask or FastAPI) for the backend and simple HTML/CSS/JavaScript for the frontend. Use the python-docx library to parse Word files.
6. Show a loading spinner while the document is being processed.
7. If no references section is found, show a helpful error message suggesting the user check that their document has a clearly labeled References section.

Prompt 1B — Refinement (use after testing 1A)

The reference extraction is working but I'd like to improve it:

1. Some references span multiple lines. Make sure the parser handles references that continue on the next line (they're usually separated by a blank line or a new number/bracket).
2. Support these common reference formats:
 - Numbered references: [1], 1., (1)
 - Author-year (APA style): Author (Year). Title...
 - Vancouver style: numbered with journal abbreviations
3. Add a count showing "Found X references" at the top of the results.

4. Add a button to download the extracted references as a CSV file with columns: Reference Number, Authors, Title, Journal, Year, DOI, Full Text.

PHASE 2: Verify References Against Real Databases

Goal: Each extracted reference gets checked against CrossRef and PubMed to confirm it's a real publication.

Prompt 2A — CrossRef Verification

Now I want to verify each extracted reference against real publication databases. Add these features:

1. After references are extracted, add a "Verify References" button.
2. When clicked, for each reference, search the CrossRef API (<https://api.crossref.org/works>) to check if the publication exists:
 - If a DOI was found in the reference, look it up directly at <https://api.crossref.org/works/{DOI}>
 - If no DOI, search by title using <https://api.crossref.org/works?query.title={title}&rows=3>
 - Compare the search results against the reference's author, year, and journal to find a match
3. Also search PubMed's free API (<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi>) by title to see if the paper exists there.
4. For each reference, show a status:
 - GREEN with a checkmark: Found an exact or strong match in CrossRef or PubMed
 - YELLOW with a warning icon: Found a partial match (title similar but some details differ)
 - RED with an X: Could not find this reference in any database — potentially fabricated
5. Show the match details: what was found in the database vs. what's in the manuscript, so users can see why something was flagged.
6. Add a summary bar at the top: "X verified, Y partial matches, Z not found"
7. Include a polite mailto: email address field at the top so that users know who is responsible for this app, and add rate limiting so we don't overwhelm the CrossRef API — add a small delay between requests. Also add the CrossRef polite pool header with an email address (I'll provide this later).
8. Show a progress bar while verification is running since it may take a while for many references.

Prompt 2B — Improve Matching Logic

The verification is working but the matching needs improvement:

1. When comparing titles, ignore case and minor punctuation differences.
2. If the CrossRef result has a DOI, display it as a clickable link so users can go directly to the paper.
3. For each verified reference, show the correct citation information from CrossRef so users can compare it against what's in their manuscript (this helps catch typos in author names, wrong years, wrong volume numbers, etc.).

4. Add a "Flag for Review" button next to each reference so users can manually mark ones they want to come back to.
 5. Add the ability to export the full verification report as a downloadable Word document or PDF, with all references listed and their verification status.
-

PHASE 3: Upload PDFs and Check Citation Accuracy

Goal: Users upload the actual PDF papers cited in the manuscript, and the app checks whether each citation in the manuscript is supported by the corresponding paper.

Prompt 3A — PDF Upload and Matching

Now I want users to be able to upload the actual PDF files of the papers they cited. Add these features:

1. Add a new section called "Upload Source Papers" below the reference verification results.
2. Users should be able to upload multiple PDF files at once (drag and drop or browse). These are the full-text papers that correspond to their references.
3. When PDFs are uploaded, the app should:
 - Extract the text from each PDF (use a Python library like PyPDF2 or pdfplumber)
 - Try to automatically match each PDF to a reference in the manuscript by comparing the PDF's title/first page against the reference list
 - Show which PDFs matched to which references, and which references still don't have a PDF uploaded
4. Show a status for each reference:
 - "PDF uploaded and matched" (with the filename)
 - "No PDF uploaded yet"
 - Allow users to manually match a PDF to a reference if the automatic matching fails (dropdown to select which reference it belongs to)
5. Allow users to upload additional PDFs later without losing previous uploads.

Prompt 3B — AI-Powered Citation Checking

Now for the most important feature. I want the app to check whether each citation in the manuscript is actually supported by the cited paper. Here's how it should work:

1. Go through the manuscript text and find every in-text citation (like [1], [2-4], (Smith 2020), etc.).
2. For each citation found in the text, extract the sentence or paragraph where it appears — this is the "claim" being made.
3. If we have the PDF for that cited reference, send both the claim and the relevant text from the PDF to an AI API to assess whether the citation supports the claim.
4. Use the Google Gemini API (model: gemini-2.0-flash) for this analysis. The prompt to the AI should be:

"You are a scientific citation checker. Below is a claim made in a manuscript and the text from the paper that was cited to support it.

CLAIM FROM MANUSCRIPT: [the sentence/paragraph with the citation]
CITED PAPER TEXT: [extracted text from the PDF]

Assess whether the cited paper supports the claim. Respond with:

- SUPPORTED: The paper clearly supports this claim
- PARTIALLY SUPPORTED: The paper touches on this topic but doesn't fully support the specific claim
- NOT SUPPORTED: The paper does not appear to support this claim
- UNCLEAR: Not enough information to determine

Provide a brief 2-3 sentence explanation of your reasoning."

5. Display results in a new "Citation Accuracy" section:

- For each in-text citation, show the claim, the referenced paper, and the AI's assessment
- Color code: green for SUPPORTED, yellow for PARTIALLY SUPPORTED, red for NOT SUPPORTED, gray for UNCLEAR
- Include the AI's explanation

6. Add a disclaimer at the top of this section: "AI-powered analysis — results should be verified by a human reviewer. This tool flags potential issues for review, it does not make definitive judgments."

7. Allow users to add their own Gemini API key in a settings area (the key should be stored securely and not visible after entry).

Prompt 3C — Citation Checking Refinements

Improve the citation checking feature:

1. For references where we don't have a PDF, the app should still check the claim against the paper's abstract (which we may have gotten from PubMed or CrossRef in Phase 2). Note when the analysis is based only on the abstract vs. the full text.
2. Add a "Check All" button that runs the AI analysis on all citations at once, with a progress bar.
3. Add a "Check This One" button next to each individual citation for spot-checking.
4. Show a summary: "X citations supported, Y partially supported, Z not supported, W unclear, V could not be checked (no PDF or abstract available)"
5. If any reference is cited multiple times in the manuscript, check each instance separately since it might support one claim but not another.

PHASE 4: Multi-User Access and Feedback

Goal: Multiple people can use the app, results are saved, and users can rate the accuracy of the checks.

Prompt 4A — User Accounts and Saving Results

Add user accounts and the ability to save results:

1. Add a simple login system with username and password. Include a registration page where new users can create an account. No email verification needed — keep it simple.
2. After login, users see a dashboard showing:
 - Their previous manuscript checks (with date, filename, and summary of results)
 - A button to start a new check
 - The ability to click into any previous check to see the full results
3. Save all results to a database (use SQLite) so they persist between sessions.
4. Each manuscript check should be saved with:
 - The filename and upload date
 - Who uploaded it (username)
 - All extracted references and their verification status
 - All citation accuracy results
 - Any user feedback (see next prompt)

Prompt 4B — Thumbs Up/Down Feedback and Admin Dashboard

Add a feedback system and admin dashboard:

1. Next to every verification result and every citation accuracy result, add thumbs up and thumbs down buttons.
 2. When a user clicks thumbs up or thumbs down, save this to the database with:
 - Which result it was for
 - Who gave the feedback
 - Timestamp
 - Optional: a small text box for comments (appears after clicking thumbs down)
 3. Create an admin dashboard (accessible only to admin users) that shows:
 - Total number of manuscripts checked
 - Total number of references verified
 - Accuracy statistics: what percentage of results got thumbs up vs thumbs down
 - A breakdown by feature (reference verification accuracy vs. citation checking accuracy)
 - A list of all users and how many manuscripts each has checked
 - The ability to export all feedback data as a CSV
 4. Make my account (username: admin) an admin account by default.
 5. On the main dashboard, show overall app statistics: "This tool has checked X manuscripts and Y references so far."
-

PHASE 5 (STRETCH GOAL): Nice-to-Have Features

Only attempt these after Phases 1–4 are solid.

Prompt 5A — Batch Processing

Add the ability to upload multiple manuscripts at once for batch processing. Show a queue with progress for each one, and let users browse results for each manuscript separately.

Prompt 5B — Team/Group Features

Add the ability to create teams. An admin can create a team, invite users by username, and all team members can see each other's manuscript checks. This way a research group can share one workspace.

Prompt 5C — Reference Format Checker

Add a feature that checks whether all references follow a consistent citation format (e.g., all APA, all Vancouver). Flag any references that don't match the majority format, and suggest corrections.

TROUBLESHOOTING TIPS

If Claude Code produces an error: Just describe the error in plain English. For example: "When I upload a Word document, I get an error that says [paste error]. Can you fix this?"

If the reference extraction misses references: Upload a sample document and say: "Here's a test document. The app found 15 references but there are actually 23. The ones it's missing are in [describe format]. Can you fix the parser?"

If the CrossRef API isn't finding papers you know exist: Say: "The reference 'Smith et al. 2020, Efficacy of X in Y patients, Journal of Z' is real but the app says it's not found. Can you improve the search logic to try different search strategies?"

If the AI citation checking seems inaccurate: Say: "The AI said this citation was not supported, but it clearly is when I read the paper. Can you adjust the AI prompt to be more nuanced and consider that the claim might be paraphrased or that the support might be indirect?"

General tip: Always test with a real manuscript you know well, so you can immediately tell if results are correct or not. Start with a short paper (10-15 references) before testing with longer ones.

API KEYS YOU'LL NEED

1. **CrossRef** — No API key needed, but add your email to the Polite Pool header for better rate limits
 2. **PubMed** — Free API key from <https://www.ncbi.nlm.nih.gov/account/> (optional but recommended)
 3. **Google Gemini** — Free API key from <https://aistudio.google.com/apikey> (free tier gives 15 requests/minute which is plenty for testing)
-

SUGGESTED TESTING APPROACH

1. Find a published paper you wrote or co-authored where you know every reference is real
2. Upload it and verify that all references come back GREEN
3. Then create a fake version with 2-3 made-up references mixed in
4. Upload the fake version and see if the app catches the fabricated ones
5. Upload the PDFs of the real cited papers and test the citation accuracy checker
6. This gives you ground truth to know if the app is working correctly